CRIANDO CÓDIGO COM PYTHON

O VS Code é um tipo especial de editor de texto chamado compilador. No topo, você notará um editor de texto. Na parte inferior, você verá um terminal onde pode executar comandos.

No terminal, você pode executar o código hello.py para começar a codificar. No editor de texto acima, você pode digitar print("olá, mundo"). Este é um programa canônico famoso que quase todos os programadores escrevem durante seu processo de aprendizagem.

Na janela do terminal, você pode executar comandos. Para rodar este programa, você precisará mover seu cursor para o final da tela, clicando na janela do terminal. Agora você pode digitar um segundo comando na janela do terminal. Ao lado do sinal de dólar, digite python hello.py e pressione a tecla enter no seu teclado. Lembre-se de que os computadores realmente só entendem zeros e uns. Portanto, quando você executa python hello.py, o python interpretará o texto que você criou em hello.py e o traduzirá para os zeros e uns que o computador pode entender.

O resultado da execução do programa python hello.py é olá, mundo. Parabéns! Você acabou de criar seu primeiro programa.

FUNÇÕES

Funções são verbos ou ações que o computador ou linguagem de computador já sabe executar.

No seu programa hello.py, a função print sabe como imprimir na janela do terminal.

A função print aceita argumentos. Neste caso, “olá, mundo” são os argumentos que a função print aceita.

BUGS

Bugs são uma parte natural da codificação. São erros, problemas para você resolver! Não desanime! Isso faz parte do processo para se tornar um grande programador.

Imagine em nosso programa hello.py que digitamos acidentalmente print("hello, world" note que perdemos o final) exigido pelo compilador. Se eu cometer esse erro propositalmente, o compilador gerará um erro na janela do terminal!

Frequentemente, as mensagens de erro informam sobre seus erros e fornecem dicas sobre como corrigi-los. No entanto, muitas vezes o compilador não será desse tipo.

MELHORANDO SEU PRIMEIRO PROGRAMA PYTHON

Podemos personalizar seu primeiro programa Python.

Em nosso editor de texto em hello.py podemos adicionar outra função. input é uma função que recebe um prompt como argumento. Podemos editar nosso código para dizer:

```
input("What's your name? ")
print("hello, world")
```

Esta edição por si só, no entanto, não permitirá que seu programa produza o que o usuário insere. Para isso, precisaremos apresentar as variáveis.

VARIÁVEIS

Uma variável é apenas um contêiner para um valor dentro do seu próprio programa.

Em seu programa, você pode introduzir sua própria variável editando-a para ler

```
name = input("What's your name? ")
print("hello, world")
```

Observe que esse sinal = de igual no meio de name = input("What´s your name?") tem um papel especial na programação. Este sinal de igual atribui literalmente o que está à direita ao que está à esquerda. Portanto, o valor retornado por input("What´s your name?") é atribuído ao name.

Se você editar seu código da seguinte maneira, notará um erro

```
name = input("What's your name? ")
print("hello, name")
```

O programa retornará hello, name na janela do terminal, independentemente do que o usuário digitar.

Editando ainda mais nosso código, você pode digitar

```
name = input("What's your name? ")
print("hello,")
```

```
print(name)
```

O resultado na janela do terminal seria

```
What's your name? David
hello
David
```

Estamos nos aproximando do resultado que pretendemos!

COMENTÁRIOS

Os comentários são uma forma de os programadores rastrearem o que estão fazendo em seus programas e até mesmo informarem outras pessoas sobre suas intenções em relação a um bloco de código. Resumindo, são notas para você e outras pessoas que verão seu código!

Você pode adicionar comentários ao seu programa para poder ver o que ele está fazendo. Você pode editar seu código da seguinte maneira:

```
# Ask the user for their name
name = input("What's your name? ")
print("hello,")
print(name)
```

Os comentários também podem servir como uma lista de tarefas para você.

PSEUDOCÓDIGO

Pseudocódigo é um tipo importante de comentário que se torna um tipo especial de lista de tarefas, especialmente quando você não entende como realizar uma tarefa de codificação. Por exemplo, no seu código, você pode editá-lo para dizer:

```
# Ask the user for their name
name = input("What's your name? ")

# Print hello
print("hello,")

# Print the name inputted
print(name)
```

MELHORANDO AINDA MAIS SEU PRIMEIRO PROGRAMA PYTHON

Podemos editar ainda mais nosso código da seguinte maneira:

```
# Ask the user for their name
name = input("What's your name? ")

# Print hello and the inputted name
print("hello, " + name)
```

Acontece que algumas funções aceitam muitos argumentos.

Podemos usar uma vírgula para passar vários argumentos editando nosso código da seguinte maneira:

```
# Ask the user for their name
name = input("What's your name? ")

# Print hello and the inputted name
print("hello,", name)
```

A saída no terminal, se digitássemos “David” seríamos hello, David. Sucesso.

STRINGS E PARÂMETROS

Uma string, conhecida como str em Python, é uma sequência de texto.

Retrocedendo um pouco em nosso código para o seguinte, houve um efeito colateral visual de fazer com que o resultado aparecesse em várias linhas:

```
# Ask the user for their name
name = input("What's your name? ")
print("hello,")
print(name)
```

As funções recebem argumentos que influenciam seu comportamento. Se olharmos a documentação do print você notará que podemos aprender muito sobre os argumentos que a função print usa.

Olhando esta documentação, você aprenderá que a função print inclui automaticamente um trecho de código end='\n'. Isto \n indica que a função de impressão criará automaticamente uma quebra de linha quando executada. A função print recebe um argumento chamado end` e o padrão é criar uma nova linha.

No entanto, podemos fornecer tecnicamente um argumento para acabarmos com nós mesmos, de modo que uma nova linha não seja criada!

Podemos modificar nosso código da seguinte maneira:

```
# Ask the user for their name
name = input("What's your name? ")
print("hello,", end="")
print(name)
```

Ao fornecer end="" estamos sobrescrevendo o valor padrão de end de forma que ele nunca crie uma nova linha após esta primeira instrução de impressão. Fornecendo o nome “David”, a saída na janela do terminal será olá, David.

Parâmetros, portanto, são argumentos que podem ser obtidos por uma função.

UM PEQUENO PROBLEMA COM ASPAS

Observe como adicionar aspas como parte de sua string é um desafio. Print("hello,"friend"") não funcionará e o compilador gerará um erro.

Geralmente, existem duas abordagens para corrigir isso. Primeiro, você poderia simplesmente alterar as aspas para aspas simples.

Outra abordagem mais comumente usada seria codificar como print("hello, \"friend\""). As barras invertidas informam ao compilador que o caractere a seguir deve ser considerado uma aspa na string e evita um erro do compilador.

FORMATANDO STRINGS

Provavelmente a maneira mais elegante de usar strings seria a seguinte:

```
# Ask the user for their name
name = input("What's your name? ")
print(f"hello, {name}")
```

Observe o f em print(f"olá, {name}"). Este f é um indicador especial para Python tratar esta string de uma maneira especial, diferente das abordagens anteriores que ilustramos nesta palestra. Espero que você use esse estilo de strings frequência nesta disciplina.

MAIS SOBRE STRINGS

Você nunca deve esperar que seu usuário coopere conforme pretendido. Portanto, você precisará garantir que a entrada do seu usuário seja corrigida ou verificada.

Acontece que embutida nas strings está a capacidade de remover espaços em branco de uma string.

Ao utilizar a faixa de método no nome como name = name.strip(), removerá todos os espaços em branco à esquerda e à direita da entrada do usuário. Você pode modificar seu código para ser:

```
# Ask the user for their name
```

```
name = input("What's your name? ")

# Remove whitespace from the str
name = name.strip()

# Print the output
print(f"hello, {name}")
```

Executar novamente este programa, independentemente de quantos espaços você digitar antes ou depois do nome, ele removerá todos os espaços em branco.

Usando o método title, o título seria o nome do usuário:

```
# Ask the user for their name
name = input("What's your name? ")

# Remove whitespace from the str
name = name.strip()

# Capitalize the first letter of each word
name = name.title()

# Print the output
print(f"hello, {name}")
```

Neste ponto, você pode estar cansado de digitar python repetidamente na janela do terminal. Você pode usar a seta para cima do teclado para recuperar os comandos de terminal mais recentes que você fez.

Observe que você pode modificar seu código para ser mais eficiente:

```
# Ask the user for their name
name = input("What's your name? ")
```

```
# Remove whitespace from the str and capitalize the first letter of each word
name = name.strip().title()

# Print the output
print(f"hello, {name}")
```

Isso cria o mesmo resultado do seu código anterior.

Poderíamos até ir mais longe!

```
# Ask the user for their name, remove whitespace from the str and capitalize the first letter of each word
name = input("What's your name? ").strip().title()

# Print the output
print(f"hello, {name}")
```

INTEIROS OU INT

Em Python, um número inteiro é chamado de int.

No mundo da matemática, estamos familiarizados com os operadores +, -, *, / e %. Esse último operador% ou módulo pode não ser muito familiar para você.

Você não precisa usar a janela do editor de texto em seu compilador para executar o código Python. No seu terminal, você pode executar o python sozinho. Você verá >>> na janela do terminal. Você pode então executar código interativo ao vivo. Você poderia digitar 1+1 e esse cálculo será executado. Este modo normalmente não será usado durante esta disciplina.

Abrindo o VS Code novamente, podemos digitar o código calculator.py no terminal. Isso criará um novo arquivo no qual criaremos nossa própria calculadora.

Primeiro, podemos declarar algumas variáveis.

```
x = 1
y = 2
```

```
z = x + y

print(z)
```

Naturalmente, quando executamos python calculator.py obtemos o resultado na janela do terminal 3. Podemos tornar isso mais interativo usando a função de entrada.

```
x = input("What's x? ")
y = input("What's y? ")

z = x + y

print(z)
```

Executando este programa, descobrimos que a saída está incorreta como 12. Por que isso acontece?

Anteriormente, vimos como o sinal + concatena duas strings. Como a entrada do teclado do computador chega ao compilador como texto, ela é tratada como uma string. Precisamos, portanto, converter essa entrada de uma string em um número inteiro. Podemos fazer isso da seguinte maneira:

```
x = input("What's x? ")
y = input("What's y? ")

z = int(x) + int(y)

print(z)
```

O resultado agora está correto. O uso de int(x) é chamado de “casting”, onde um valor é temporariamente alterado de um tipo de variável (neste caso, uma string) para outro (aqui, um inteiro).

Podemos melhorar ainda mais nosso programa da seguinte forma:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

print(x + y)
```

Isso ilustra que você pode executar funções em funções. A função interna é executada primeiro e depois a externa. Primeiro, a função de entrada é executada. Então, a função int.

A LEGIBILIDADE VENCE

Ao decidir sobre sua abordagem para uma tarefa de codificação, lembre-se de que é possível apresentar um argumento razoável para muitas abordagens para o mesmo problema.

Independentemente da abordagem adotada para uma tarefa de programação, lembre-se de que seu código deve ser legível. Você deve usar comentários para dar a si mesmo e aos outros pistas sobre o que seu código está fazendo. Além disso, você deve criar o código de uma forma que seja legível.

NOÇÕES BÁSICAS DE FLOAT

Um valor de ponto flutuante é um número real que contém uma vírgula decimal, como 0,52.

Você pode alterar seu código para suportar floats da seguinte maneira:

```
x = float(input("What's x? "))
y = float(input("What's y? "))

print(x + y)
```

Esta alteração permite que seu usuário insira 1.2 e 3.4 para apresentar um total de 4.6.

Vamos imaginar, porém, que você deseja arredondar o total para o número inteiro mais próximo. Olhando a documentação do Python para round, você verá que os argumentos disponíveis são round(number[n, ndigits]). Esses colchetes indicam que algo opcional pode ser especificado pelo programador. Portanto,

você poderia fazer round(n) para arredondar um dígito para o número inteiro mais próximo. Alternativamente, você poderia codificar da seguinte forma:

```
# Get the user's input
x = float(input("What's x? "))
y = float(input("What's y? "))

# Create a rounded result
z = round(x + y)

# Print the result
print(z)
```

A saída será arredondada para o número inteiro mais próximo.

E se quiséssemos formatar a saída de números longos? Por exemplo, em vez de ver 1.000, você pode querer ver 1.000. Você poderia modificar seu código da seguinte maneira:

```
# Get the user's input
x = float(input("What's x? "))
y = float(input("What's y? "))

# Create a rounded result
z = round(x + y)

# Print the formatted result
print(f"{z:,}")
```

Embora bastante enigmático, esse print(f"{z:,}") cria um cenário onde o z gerado incluirá vírgulas onde o resultado pode parecer 1.000 ou 2.500.

MAIS SOBRE FLOATS

Como podemos arredondar valores de ponto flutuante? Primeiro, modifique seu código da seguinte maneira:

```
# Get the user's input
x = float(input("What's x? "))
y = float(input("What's y? "))

# Calculate the result
z = x / y

# Print the result
print(z)
```

Ao inserir 2 como x e 3 como y, o resultado z é 0,6666666666, aparentemente indo para o infinito como poderíamos esperar.

Vamos imaginar que queremos arredondar isso para baixo. Poderíamos modificar nosso código da seguinte maneira:

```
# Get the user's input
x = float(input("What's x? "))
y = float(input("What's y? "))

# Calculate the result and round
z = round(x / y, 2)

# Print the result
print(z)
```

Como seria de esperar, isso arredondará o resultado para as duas casas decimais mais próximas.

Também poderíamos usar fstring para formatar a saída da seguinte forma:

```
# Get the user's input
x = float(input("What's x? "))
```

```
y = float(input("What's y? "))

# Calculate the result
z = x / y

# Print the result
print(f"{z:.2f}")
```

Este código fstring enigmático exibe o mesmo que nossa estratégia de arredondamento anterior.

FUNÇÕES DEF

Não seria bom criar nossas próprias funções?

Vamos trazer de volta nosso código final hello.py digitando o código hello.py na janela do terminal. Seu código inicial deve ficar assim:

```
# Ask the user for their name, remove whitespace from the str and capitalize the first letter of each word
name = input("What's your name? ").strip().title()

# Print the output
print(f"hello, {name}")
```

Podemos melhorar nosso código para criar nossa própria função especial que diz “olá” para nós!

Apagando todo o nosso código no nosso editor de texto, vamos começar do zero:

```
name = input("What's your name? ")
hello()
print(name)
```

Ao tentar executar este código, seu compilador gerará um erro. Afinal, não existe uma função definida para hello.

Podemos criar nossa própria função chamada hello da seguinte maneira:

```
def hello():
    print("hello")


name = input("What's your name? ")
hello()
print(name)
```

Observe que tudo em def hello() está recuado. Python é uma linguagem recuada. Ele usa recuo para entender o que faz parte da função acima. Portanto, tudo na função hello deve ser recuado. Quando algo não está recuado, ele trata como se não estivesse dentro da função hello. Executando python hello.py na janela do terminal, você verá que sua saída não é exatamente como você deseja.

Podemos melhorar ainda mais nosso código:

```
# Create our own function
def hello(to):
    print("hello,", to)


# Output using our own function
name = input("What's your name? ")
hello(name)
```

Aqui, nas primeiras linhas, você está criando sua função hello. Desta vez, porém, você está informando ao compilador que esta função recebe um único parâmetro: uma variável chamada to. Portanto, quando você chama hello(name), o computador passa name para a função hello. É assim que passamos valores para funções. Muito útil! Executando python hello.py na janela do terminal, você verá que a saída está muito mais próxima do nosso ideal apresentado anteriormente nesta palestra.

Podemos alterar nosso código para adicionar um valor padrão a hello:

```
# Create our own function
def hello(to="world"):
    print("hello,", to)
```

```
# Output using our own function
name = input("What's your name? ")
hello(name)

# Output without passing the expected arguments
hello()
```

Teste seu código você mesmo. Observe como o primeiro hello se comportará conforme o esperado, e o segundo hello, ao qual não é passado um valor, irá, por padrão, gerar hello, world.

Não precisamos ter nossa função no início do nosso programa. Podemos movê-lo para baixo, mas precisamos informar ao compilador que temos uma função principal e uma função hello separada.

```
def main():

    # Output using our own function
    name = input("What's your name? ")
    hello(name)

    # Output without passing the expected arguments
    hello()


# Create our own function
def hello(to="world"):
    print("hello,", to)
```

Isso por si só, entretanto, criará uma espécie de erro. Se executarmos python hello.py, nada acontece! A razão para isso é que nada neste código está realmente chamando a função principal e dando vida ao nosso programa.

A seguinte pequena modificação chamará a função principal e restaurará nosso programa ao funcionamento:

```
def main():

    # Output using our own function
    name = input("What's your name? ")
    hello(name)

    # Output without passing the expected arguments
    hello()


# Create our own function
def hello(to="world"):
    print("hello,", to)

main()
```

RETORNANDO VALORES

Você pode imaginar muitos cenários em que não deseja apenas que uma função execute uma ação, mas também retorne um valor para a função principal. Por exemplo, em vez de simplesmente imprimir o cálculo de x + y, você pode querer que uma função retorne o valor desse cálculo para outra parte do seu programa. Chamamos esse “repasse” de um valor de valor de retorno.

Retornando ao nosso código calculator.py digitando o código calculator.py. Apague todo o código lá. Refaça o código da seguinte maneira:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))


def square(n):
```

```
    return n * n


main()
```

Efetivamente, x é passado para o quadrado. Então, o cálculo de x * x é retornado à função principal.

RESUMINDO

Através do trabalho desta única palestra, você aprendeu habilidades que usará inúmeras vezes em seus próprios programas. Você aprendeu sobre…

- Criando seus primeiros programas em Python;
- Funções;
- Bugs;
- Variáveis;
- Comentários;
- Pseudo-código;
- Strings;
- Parâmetros;
- Formatando Strings;
- Inteiros;
- Float;
- Criando suas próprias funções;
- Valores de retorno.